Project – Milestone 3

Contents

Group members....2
Member Roles....3
Chosen Programming Language....3
Communication Method....3
Project Gantt Chart....4
GitHub Link....5
Milestone 1 Problem Solutions....5
Problem Statement 1....5
Problem Statement 2....7
Problem Statement 3....8
Solutions....10
Task 1 – Problem 1 Brute Force....10
Input....10
Output....10
Constraints....10
Pseudocode....11
Python Code....11
Task 2 – Problem 1 Greedy Solution....12
Input....12
Output....13
Constraints....13
Pseudocode....13
Python Code....14
Task 3 – Dynamic Programming Algorithm for Problem 1....15
Input....15
Output....15

Constraints 16
Pseudocode 16
Python Code. 17
Task – 6: Dynamic Programming Algorithm for Problem 3 18
Input 18
Output 18
Constraints 18
Pseudocode 19
Python Code. 21
Task – 7: Dynamic Programming Algorithm for Problem 3 23
Input 23
Output 23
Constraints 24
Pseudocode 24
Python Code. 25

Group members

Austin Hall
email: austinhall@ufl.edu
GitHub: aahall05

Stephen Kramer
email: kramers@ufl.edu
GitHub: sokramerz

Dajana Seitllari
Email: seitllarid@ufl.edu
Github: seitllarid

Richard Huang
Email: rhuang3@ufl.edu
Github: richardhuang12

Member Roles

Austin Hall – Product Manager

Stephen Kramer – Scrum Master

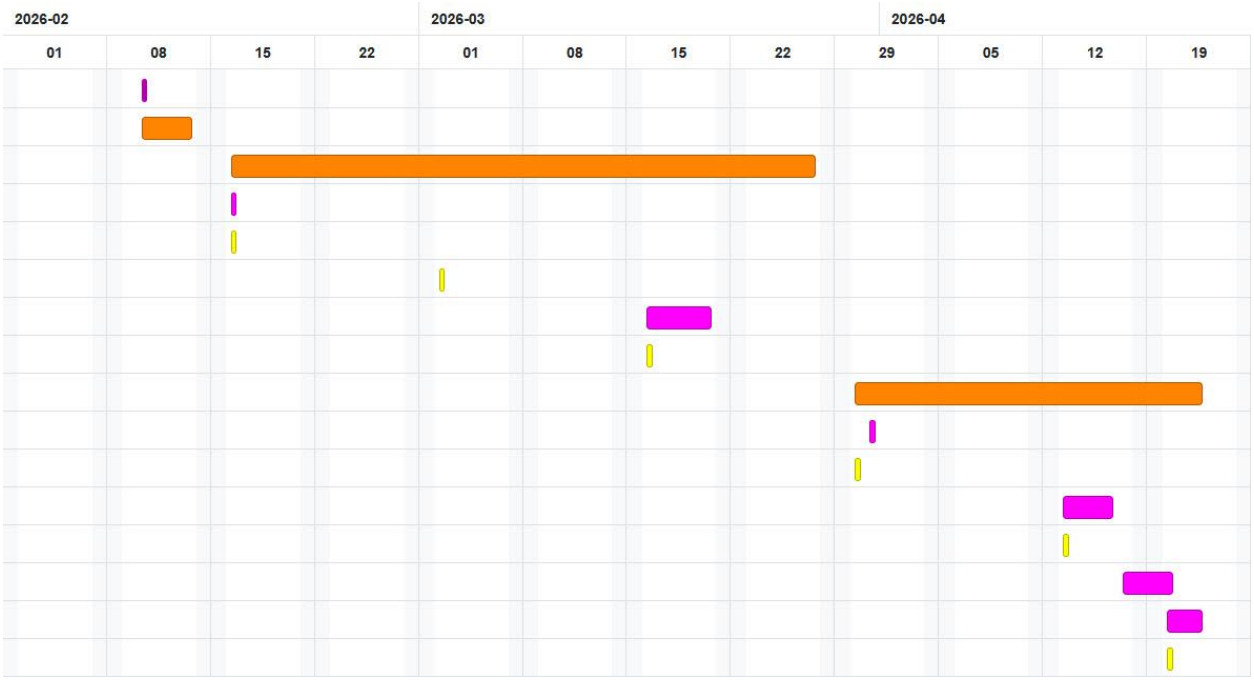
Chosen Programming Language

Our group has decided to implement our project using Python

Communication Method

The group created a Discord server to communicate. The discord contains our emails and phone numbers for emergency situations.

Project Gantt Chart



ID	Task Name	Start	End	Duration	Color
1	Project file and Github creation	2026-02-10	2026-02-10	1 day	
2	Milestone 1	2026-02-10	2026-02-13	4 days	
3	Milestone 2	2026-02-16	2026-03-27	30 days	
8	MS2 - Implementation planning	2026-02-16	2026-02-16	1 day	
4	Team Meeting 1	2026-02-16	2026-02-16	1 day	
5	Team Meeting 2	2026-03-02	2026-03-02	1 day	
9	MS2 - Implementation Design completed	2026-03-16	2026-03-20	5 days	
6	Team Meeting 3	2026-03-16	2026-03-16	1 day	
11	Milestone 3	2026-03-30	2026-04-22	18 days	
12	MS3 - Planning	2026-03-31	2026-03-31	1 day	
7	Team Meeting 4	2026-03-30	2026-03-30	1 day	
16	MS3 - Implementation Finished	2026-04-13	2026-04-16	4 days	
14	Team Meeting 5	2026-04-13	2026-04-13	1 day	
10	MS3 - Testing	2026-04-17	2026-04-20	2 days	
15	Project Finalization	2026-04-20	2026-04-22	3 days	
17	Team Meeting 6	2026-04-20	2026-04-20	1 day	

GitHub Link

https://github.com/aahall05/COP4533-Project-Group-6.git

Milestone 1 Problem Solutions

Problem Statement 1

1. $A = \begin{matrix} 12 & 1 & 5 & 3 & 16 \\ 4 & 4 & 13 & 4 & 9 \\ 6 & 8 & 6 & 1 & 2 \\ 14 & 3 & 4 & 8 & 10 \end{matrix}$
2. Profit for each sale is calculated using $(sell\ price) - (price\ paid)$
    a. Limitations:
        i. If bought and sold on the same day, profit is zero
        ii. The last stock can only be bought and sold on the last day
        iii. A stock bought on one day, can not be sold on any days prior
    b. For the first stock:
        i. Buying on day one, the profits for selling each subsequent day would be:
            1. $-11 \quad -7 \quad -9 \quad 4$
        ii. Buying on day two, the profits for selling each subsequent day would be:
            1. $4 \quad -2 \quad 15$
        iii. Buying on day three, the profits for selling each subsequent day would be:
            1. $-2 \quad 11$
        iv. Buying on day four, the profit for selling each subsequent day would be:
            1. 13
        v. Buying on day five, the profit would be 0 since it must be sold that day
    c. For the second stock:
        i. Buying on day one, the profits for selling each day would be:
            1. $0 \quad 9 \quad 0 \quad 5$
        ii. Buying on day two, the profits for selling each day would be:
            1. $9 \quad 0 \quad 5$
        iii. Buying on day three, the profits for selling each day would be:
            1. $-9 \quad -4$
        iv. Buying on day four, the profit for selling each day would be:

1. 5

v. Buying on day four, the profit would be 0 since it must be sold that day

d. For the third stock:

i. Buying on day one, the profits for selling each subsequent day would be:

1. $2 \quad 0 \quad -5 \quad -4$

ii. Buying on day two, the profits for selling each subsequent day would be:

1. $-2 \quad -7 \quad -6$

iii. Buying on day three, the profits for selling each subsequent day would be:

1. $-5 \quad -4$

iv. Buying on day four, the profit for selling each subsequent day would be:

1. -1

v. Buying on day five, the profit would be 0 since it must be sold that day

e. For the fourth stock:

i. Buying on day one, the profits for selling each day would be:

1. $-11 \quad -10 \quad -6 \quad -4$

ii. Buying on day two, the profits for selling each day would be:

1. $1 \quad 5 \quad 7$

iii. Buying on day three, the profits for selling each day would be:

1. $4 \quad 6$

iv. Buying on day four, the profit for selling each day would be:

1. 2

v. Buying on day four, the profit would be 0 since it must be sold that day

3. For each stock, the best day to buy and sell is: in the format (buy day, sell day)=profit
   a. Stock 1: $(2,5) = 15$
   b. Stock 2: $(1,3) = 9$
   c. Stock 3: $(1,2) = 2$
   d. Stock 4: $(2,5) = 7$
4. So the stock, and the days to buy and sell it, with the highest profit is stock one, bought on the second day and sold on the fifth day
   a. $(1,2,5,15)$

First we will find the best days to buy and sell each stock to maximize profit

Problem Statement 2

$$A = \begin{matrix} 25 & 30 & 15 & 40 & 50 \\ 10 & 20 & 30 & 25 & 5 \\ 30 & 45 & 35 & 10 & 15 \\ 5 & 50 & 35 & 25 & 45 \end{matrix}$$

I = 4 different stocks

J = 5 days

K = Constraint of maximum of 3 transactions

1. First we calculate potential profit for single-day interval.
    a. To determine the profit, simply subtract the earlier buy price from the later sell price. For Stock 1, we subtract the Day 1 price $25 at A{1,1} from the Day 2 price $30 at A{1,2}, resulting in a profit of $5.

- **Day 1 → 2**
    - Stock 1: $30 – $25 = $5
    - Stock 2: $20 – $10 = $10
    - Stock 3: $45 - $30 = $15
    - Stock 4: $50 - $5 = $45
    - **Optimal:** Stock 4 (Profit: 45)
- **Day 2 → Day 3**
    - Stock 1: $15 - $30 = -$15
    - Stock 2: $30 - $20 = $10
    - Stock 3: $35 - $45 = -$10
    - Stock 4: $35 - $50 = -$15
    - **Optimal:** Stock 2 (Profit: 10)
- **Day 3 → Day 4**
    - Stock 1: $40 - $15 = $25
    - Stock 2: $25 - $30 = -$5
    - Stock 3: $10 - $35 = -$25
    - Stock 4: $25 - $35 = -$10
    - **Optimal**: Stock 1 (Profit: 25)
- **Day 4 → Day 5**
    - Stock 1: $50 - $40 = 10
    - Stock 2: $5 - $25 = -$20$
    - Stock 3: $15 - $10 = 5

- Stock 4: $45 - $25 = $20
- **Optimal:** Stock 4 (Profit: 20)

b. Now that we've done the calculations for each day, we can select the optimal transactions which are the most profitable:

   i. **Day 1-2:** Profit **45** (Stock 4)
   ii. **Day 2-3:** Profit **10** (Stock 2)
   iii. **Day 3-4:** Profit **25** (Stock 1)
   iv. **Day 4-5:** Profit **20** (Stock 4)

c. We are limited to 3 transactions per the problem statement (k = 3), so we just chose the three that are the most profitable, which gives a **total profit of $90.**

2. Applying these methods to Matrix A, we get this transaction sequence:
   a. Stock 4:
      i. Buy Day 1, Sell Day 2 = $45 Profit
   b. Stock 1:
      i. Buy Day 3, Sell Day 4 = $25 Profit
   c. Stock 4: Buy Day 4, Sell Day 5 = $20 Profit

3. This can be expressed in the (I, j, l) format given as:
   a. [(4, 1, 2), (1, 3, 4), (4, 4, 5)]

Problem Statement 3

1. $A = \begin{matrix} 7 & 1 & 5 & 3 & 6 & 8 & 9 \\ 2 & 4 & 3 & 7 & 9 & 1 & 8 \\ 5 & 8 & 9 & 1 & 2 & 3 & 10 \\ 9 & 3 & 4 & 8 & 7 & 4 & 1 \\ 3 & 1 & 5 & 8 & 9 & 6 & 4 \end{matrix}$ c =2
2. With c = 2, in order to perform two transactions, the first stock must be bought on day one or two
   a. If a stock is bought on day one, it must be sold on day two or day three
   b. If a stock is bought on day two, it must be sold on day three.
3. This gives us three options for the first stock:
   a. (n, 1, 2)
   b. (n, 1, 3)
   c. (n, 2, 3)
4. The profits for option a:

    a. (1, 1, 2) = 1 – 7 = -6
    b. (2, 1, 2) = 4 – 2 = 2
    c. (3, 1, 2) = 8 - 5 = 3
    d. (4, 1, 2) = 3 – 9 = -6
    e. (5, 1, 2) = 1 – 3 = -2
5. The profits for option b:
    a. (1, 1, 3) = 5 – 7 = -2
    b. (2, 1, 3) = 3 – 2 = 1
    c. (3, 1, 3) = 9 - 5 = 4
    d. (4, 1, 3) = 4 – 9 = -5
    e. (5, 1, 3) = 5 – 3 = 2
6. The profits for option c
    a. (1, 2, 3) = 5 - 1 = 4
    b. (2, 2, 3) = 3 - 4= -1
    c. (3, 2, 3) = 9 - 8 = 1
    d. (4, 2, 3) = 4 - 3 = 1
    e. (5, 2, 3) = 5 - 1 = 4
7. The max profit of all these options is 4. This is achieved through multiple trades: (3, 1, 3), (1, 2, 3), (5, 2, 3). We will choose the first available option: (3, 1, 3)
8. With that trade being our first, we cannot buy another stock for c = 2 days.
9. This means our second trade must be bought on day 6 and sold on day 7
10. The possible profits for those specific buy/sell days are:
    a. (1, 6, 7) = 9 - 8 = 1
    b. (2, 6, 7) = 8 - 1 = 7
    c. (3, 6, 7) = 10 - 3 = 7
    d. (4, 6, 7) = 1 - 4 = -3
    e. (5, 6, 7) = 6 - 4 = 2
11. The maximum profit possible for that second day is 7 and can be achieved using multiple trades: (2, 6, 7), (3, 6, 7)
12. Taking the first trade, we have (2, 6, 7)
13. The two trades that maximize profit are
    a. (3, 1, 3) = 4
    b. (2, 6, 7) = 7
14. So adding these together we get 11 profit
15. The best trade sequence is [(3, 1, 3), (2, 6, 7)] = 11

Solutions

IMPORTANT CHANGE TO ALL IMPLEMENTATIONS

The pseudocode in Milestone 2 was written using 1-based indexing for explanation purposes. In this implementation, we have switched to 0-based indexing FOR ALL tasks, as it since it's the natural convention in Python and simplifies implementation. All inputs, outputs, and test cases are written with this in mind.

Task 1 – Problem 1 Brute Force

Input

A matrix A of dimensions $m \times n$ $(1 \leq m, n \leq 1000)$, where each element $A[i][j]$ $(0 \leq A[i][j] \leq 10^5)$ represents the predicted price of the i-th stock on the j-th day. Rows represent stocks, columns represent days.

Output

Return a tuple $(i, j_1, j_2, profit)$ where:

• i $(1 \leq i \leq m)$ is the index of the chosen stock.

• j_1 $(1 \leq j1 \leq n)$ is the day when you would buy the stock to maximize profit.

• j_2 $(1 \leq j_2 \leq n)$ is the day when you would sell the stock to maximize profit.

• Profit is the maximum profit achievable through this single transaction (sell price minus buy price)

Constraints

1. You can only perform one transaction, which involves buying and selling a single stock. You can only sell the stock you bought previously.

2. You must buy before selling; in other words, j_1 must be less than j_2.

3. The prices are non-negative integers and represent the stock's value at a specific time.

4. You want to maximize profit, so you're looking for the highest possible difference between the sell price and the buy price.

5. If no profitable transaction is possible, return a tuple (0, 0, 0, 0) to indicate this.

6. This will be a **brute force** algorithm with a time complexity of $O(m * n^2)$ using 1-based indexing

Pseudocode

We will have an int array variable called `result` that will store the information about the stock with the largest profit found, initialized to zeros for the base case.

`result[stock, buy date, sell date, profit]`

There will be three variables: `i` tracks the stock, `j1` tracks the buying day, and `j2` tracks the selling day.

```
result = [0, 0, 0, 0]
for each stock i from 1 to m
     for each buy day j1 from 1 to n - 1
          for each sell day j2 from j1 + 1 to n
               profit = A[i][j2] - A[i][j1]
               if profit > result.profit
                    result = [i, j1, j2, profit]
return result
```

The time complexity of the algorithm is O(m * n2) because the outer loop is O(m), and the inner loops are O(n), making the total complexity O(m) * O(n) * O(n) = O(m * n2)

Python Code

```
def task1(A):
  # PROBLEM 1: BRUTE FORCE


  m = len(A)
```

```
n = len(A[0])



# result stores the information about the stock with the largest profit

# found, initialized to zeros for the base case.

# result = [stock, buy date, sell date, profit]

result = [0, 0, 0, 0]



# i tracks the stock, j1 tracks the buying day, j2 tracks the selling day

# loop through every possibility, calculating profit and recording max along the way

for i in range(m):

  for j1 in range(n - 1):

    for j2 in range(j1 + 1, n):

      profit = A[i][j2] - A[i][j1]

      if profit > result[3]:

        result = [i, j1, j2, profit]



return tuple(result)
```

Task 2 – Problem 1 Greedy Solution

Input

A matrix A of dimensions $m \times n$ $(1 \leq m, n \leq 1000)$, where each element $A[i][j]$ $(0 \leq A[i][j] \leq 10^5)$ represents the predicted price of the i-th stock on the j-th day. Rows represent stocks, columns represent days.

Output

Return a tuple $(i, j_1, j_2, profit)$ where:

• i $(1 \leq i \leq m)$ is the index of the chosen stock.

• j_1 $(1 \leq j1 \leq n)$ is the day when you would sell the stock to maximize profit.

• j_2 $(1 \leq j_2 \leq n)$ is the day when you would sell the stock to maximize profit.

• profit is the maximum profit achievable through this single transaction (sell price minus buy price).

Constraints

1. You can only perform one transaction, which involves buying and selling a single stock. You can only sell the stock you bought previously.

2. You must buy before selling; in other words, j_1 must be less than j_2.

3. The prices are non-negative integers and represent the stock's value at a specific time.

4. You want to maximize profit, so you're looking for the highest possible difference between the sell price and the buy price.

5. If no profitable transaction is possible, return a tuple (0, 0, 0, 0) to indicate this.

6. This will be a **greedy** algorithm with a time complexity of $O(m * n^2)$ using 1-based indexing

Pseudocode

For each stock, this algorithm will track the lowest price for that stock, and as we move through n, calculate the profit if we were to sell on that day.

We will have the same result variable as in task 1 that will store the information about the stock with the largest profit found, initialized to zeros for the base case.

```
result[stock, buy date, sell date, profit]
```

We will also have a variable to track the lowest price, `lowest_price`, an array to track its index, `LP_index`, a variable `i` to track the outer stock loop, and variable `j` to track the inner price loop.

```
result = [0, 0, 0, 0]
```

```
for each stock i from 1 to m
     lowest_price = A[i][1];
     LP_index = 1;
     for each sell day j from 2 to n – 1
          profit = A[i][j] - A[i][LP_index]
          if profit > result.profit
               result = [i, LP_index, j, profit]
          if A[i][j] < lowest_price
               lowest_price = A[i][j]
               LP_index = j
return result
```

Python Code

```
def task2(A):
  # PROBLEM 1: GREEDY
  m = len(A)
  n = len(A[0])
  # result stores the information about the stock with the largest profit
  # found, initialized to zeros for the base case.
  # result = [stock, buy date, sell date, profit]
  result = [0, 0, 0, 0]
  # i tracks the outer stock loop, j tracks the inner price loop
  # lowest_price tracks the lowest price seen, LP_index tracks its day
```

```
    for i in range(m):
        # initialize lowest price and its index to first day
        lowest_price = A[i][0]
        LP_index = 0
        for j in range(1, n):
            # calculate profit if we were to sell on day j
            profit = A[i][j] - lowest_price
            # if this is the best profit so far, update result
            if profit > result[3]:
                result = [i, LP_index, j, profit]
            # update lowest price and its index if this is a cheaper buy day
            if A[i][j] < lowest_price:
                lowest_price = A[i][j]
                LP_index = j

    return tuple(result)
```

Task 3 – Dynamic Programming Algorithm for Problem 1

Input

A matrix A of dimensions $m \times n$ ($1 \leq m, n \leq 1000$), where each element $A[i][j]$ ($0 \leq A[i][j] \leq 05$) represents the predicted price of the i-th stock on the j-th day. Rows represent stocks, columns represent days.

Output

Return a tuple $(i, j1, j2, profit)$ where:

- i ($1 \leq i \leq m$) is the index of the chosen stock.

• $j1$ ($1 \leq j1 \leq n$) is the day when you would buy the stock to maximize profit.

• $j2$ ($1 \leq j2 \leq n$) is the day when you would sell the stock to maximize profit.

• profit is the maximum profit achievable through this single transaction (sell price minus buy price).

Constraints

1. You can only perform one transaction, which involves buying and selling a single stock. you can only sell the stock you bought previously.

2. You must buy before selling; in other words, $j1$ must be less than $j2$.

3. The prices are non-negative integers and represent the stock's value at a specific time.

4. You want to maximize profit, so you're looking for the highest possible difference between the sell price and the buy price.

5. If no profitable transaction is possible, return a tuple (0, 0, 0, 0) to indicate this.

6. This will be a dynamic programming algorithm with a time complexity of O(m * n) using 1 based indexing.

Pseudocode

```
We'll use a result array to store the best stock, buy day, sell
day, and profit found.
result = [0, 0, 0, 0]
Three variables: i tracks the stock, j tracks the day, and
min_day tracks the day with the lowest price seen so far.
for each stock i from 0 to m-1
    min_day = 0
    for each day j from 1 to n-1
        profit = A[i][j] - A[i][min_day]
        if profit > result[3]
            result = [i, min_day, j, profit]
```

```
        if A[i][j] < A[i][min_day]
            min_day = j
return result
```

The time complexity is O(m * n): the outer loop is O(m) and the inner loop is O(n), giving O(m) * O(n) = O(m * n).

Python Code.

```
def task3(A):
    # PROBLEM 1: DYNAMIC PROGRAMMING
    # Uses 0-based indexing throughout
    m = len(A)
    n = len(A[0])

    # result stores the best [stock, buy day, sell day, profit] found
    result = [0, 0, 0, 0]

    # i tracks the stock, j tracks the day
    # min_day tracks the day with the lowest price seen so far for this stock
    for i in range(m):
        min_day = 0
        for j in range(1, n):
            profit = A[i][j] - A[i][min_day]
```

```
            # update result if this is the best profit seen
            if profit > result[3]:
                result = [i, min_day, j, profit]
            # update min_day if today is cheaper than the
current minimum
            if A[i][j] < A[i][min_day]:
                min_day = j


    return tuple(result)
```

Task – 6: Dynamic Programming Algorithm for Problem 3

Input

- Matrix A: A matrix of dimensions m x n, where each element $A[i][j]$ represents the predicted price of the $i-th$ stock on the $j-th$ day.
- Integer c: The cooldown period needed after selling a stock before another

purchase can be made ($1 \le c \le n-2$).

Output

Sequence of tuples $(i, j1, j2)$ representing the transactions:

- i: The index of the chosen stock ($1 \le i \le m$).
- $j1$: The day the stock was purchased.
- $j2$: The day the stock was sold to maximize profit.
- The goal is to maximize the sum of profits across all transactions while adhering to the cooldown constraint.

Constraints

1. Multiple Transactions: You can perform multiple transactions but can only hold one stock at any given time.

2. Cooldown Requirement: If you sell a stock on day j, you can't buy any stock until day j + c + 1.

3. Order of Operations: You have to buy before selling; therefore, $j1$ must be strictly less than $j2$.

4. No Profitable Trades: If no profit is possible, return an empty sequence.

5. Complexity: This algorithm uses a Top-Down Recursive approach with memorization to achieve a time complexity of $O(m * n2)$ using 1-based indexing.

Pseudocode

```
Uses a Top-Down Recursive approach with memoization. O(m*n^2)
time, O(n) space.
Algorithm solveMaxProfit(priceMatrix, cooldownPeriod):
    stockCount = number of rows in priceMatrix
    totalDays  = number of columns in priceMatrix

    // memoMap[d] stores (maxProfit, transactionList) for
starting day d
    memoMap = array of size totalDays, initialized to null

    Function computeBestStartingAt(currentDay):
        // Base case: cannot trade past the last day
        if currentDay >= totalDays:
            return (0, [])

        // Return cached result if available
        if memoMap[currentDay] is not null:
```

```
            return memoMap[currentDay]


        // CHOICE A: skip currentDay, look for a better start
tomorrow
        (maxProfit, bestTxns) =
computeBestStartingAt(currentDay + 1)


        // CHOICE B: buy on currentDay, sell on some future day
k
        for k from (currentDay + 1) to totalDays - 1:
            for i from 0 to stockCount - 1:
                tradeProfit = priceMatrix[i][k] -
priceMatrix[i][currentDay]
                if tradeProfit > 0:
                    // Enforce cooldown: next buy cannot occur
before day k + c + 1
                    nextAvailableDay = k + cooldownPeriod + 1
                    (futureProfit, futureTxns) =
computeBestStartingAt(nextAvailableDay)
                    currentTotal = futureProfit + tradeProfit
                    if currentTotal > maxProfit:
                        maxProfit = currentTotal
                        bestTxns  = [(i, currentDay, k)] +
```

```
futureTxns


        // Cache and return
        memoMap[currentDay] = (maxProfit, bestTxns)
        return memoMap[currentDay]


    // Kick off recursion from day 0 (0-based indexing)
    (finalProfit, finalSequence) = computeBestStartingAt(0)
    return finalSequence
```

Python Code.

```
# PROBLEM 3: DYNAMIC PROGRAMMING
# CHANGED: these variables were parameters in the pseudocode,
#       but are extracted from the matrix here
stockCount = len(priceMatrix)
totalDays = len(priceMatrix[0])


# memoMap stores [maxProfit, transactionList] for each starting day
memoMap = [None] * totalDays


def computeBestStartingAt(currentDay):
  # base case: cannot trade if we are past the last day
```

```
if currentDay >= totalDays:
    return (0, [])

# check if we have already solved this sub-problem
if memoMap[currentDay] is not None:
    return memoMap[currentDay]

# CHOICE A: don't buy anything on currentDay
# we just move to the next day to see if a better deal starts there
maxProfit, bestTxns = computeBestStartingAt(currentDay + 1)

# CHOICE B: buy a stock on currentDay and sell it on some future day k
for k in range(currentDay + 1, totalDays):
    for i in range(stockCount):
        # calculate profit for this specific single trade
        tradeProfit = priceMatrix[i][k] - priceMatrix[i][currentDay]

        if tradeProfit > 0:
            # after selling on day k, we jump ahead by (k + cooldownPeriod + 1)
            # this satisfies the "wait c days" rule
            nextAvailableDay = k + cooldownPeriod + 1
            futureProfit, futureTxns = computeBestStartingAt(nextAvailableDay)
```

```
            currentTotal = futureProfit + tradeProfit



            # if this path is better, update our best result for this day

            if currentTotal > maxProfit:

                maxProfit = currentTotal

                bestTxns = [(i, currentDay, k)] + futureTxns


    # cache the result before returning

    memoMap[currentDay] = (maxProfit, bestTxns)

    return memoMap[currentDay]


# kick off the recursion starting at day 0

finalProfit, finalSequence = computeBestStartingAt(0)

return finalSequence
```

Task – 7: Dynamic Programming Algorithm for Problem 3

Input

A matrix A of dimensions m × n ($1 \le m,n \le 1000$), where each element A[i][j] ($0 \le A[i][j] \le 10^5$) represents the predicted price of the i-th stock on the j-th day.

An integer c ($1 \le c \le n - 2$) representing the cooldown period after selling.

Output

Sequence of tuples $(i, j1, j2)$ representing the transactions:

- i: The index of the chosen stock ($1 \le i \le m$).

- $j1$: The day the stock was purchased.
- $j2$: The day the stock was sold to maximize profit.
- The goal is to maximize the sum of profits across all transactions while adhering to the cooldown constraint.

Constraints

1. Multiple Transactions: You can perform multiple transactions but can only hold one stock at any given time.

2. Cooldown Requirement: If you sell a stock on day j, you can't buy any stock until day $j + c + 1$.

3. Order of Operations: You have to buy before selling; therefore, $j1$ must be strictly less than $j2$.

4. No Profitable Trades: If no profit is possible, return an empty sequence.

5. Complexity: This algorithm uses a Bottom-Up Recursive approach with memorization to achieve a time complexity of $O(m * n)$ using 1-based indexing.

Pseudocode

```
Uses a Bottom-Up DP approach. O(m*n) time, O(n) space.
Algorithm solveMaxProfit(priceMatrix, cooldownPeriod):
    stockCount = number of rows in priceMatrix
    totalDays  = number of columns in priceMatrix

    // dp[j]       = max profit achievable up to and including
day j
    // txnTrack[j] = transaction list that achieves dp[j]
    dp       = array of size totalDays, initialized to 0
    txnTrack = array of size totalDays, each initialized to []

    for i from 0 to stockCount - 1:
        // best_buy tracks the lowest effective cost seen so
far for stock i
        best_buy     = priceMatrix[i][0]
        best_buy_day = 0

        for j from 1 to totalDays - 1:
            // Update best_buy using the cooldown constraint
            if j - cooldownPeriod - 1 >= 0:
                candidate = priceMatrix[i][j] - dp[j -
cooldownPeriod - 1]
            else:
                candidate = priceMatrix[i][j]
```

```
            if candidate < best_buy:
                best_buy     = candidate
                best_buy_day = j

            profit = priceMatrix[i][j] - best_buy

            // Option 1: skip today (carry forward yesterday's
best)
            if dp[j - 1] >= dp[j]:
                dp[j]       = dp[j - 1]
                txnTrack[j] = txnTrack[j - 1]

            // Option 2: sell today (if profitable)
            if profit > 0:
                if j - cooldownPeriod - 1 >= 0:
                    prevTxns  = txnTrack[j - cooldownPeriod -
1]
                    newProfit = profit + dp[j - cooldownPeriod
- 1]
                else:
                    prevTxns  = []
                    newProfit = profit
                if newProfit > dp[j]:
                    dp[j]       = newProfit
                    txnTrack[j] = prevTxns + [(i, best_buy_day,
j)]

    return txnTrack[totalDays - 1]
```

Python Code.

```
# PROBLEM 3: DYNAMIC PROGRAMMING
# CHANGED: these variables were parameters in the pseudocode,
#       but are extracted from the matrix here
stockCount = len(priceMatrix)
totalDays = len(priceMatrix[0])


# dp[j] stores the max profit achievable up to day j
dp = [0] * totalDays

# txnTrack[j] stores the transaction sequence that gives dp[j]
txnTrack =  []
for _ in range(totalDays):
```

```
    txnTrack.append([])

# Process each stock independently
for i in range(stockCount):
    # Track best effective buy:
    # (price - previous profit before cooldown)
    best_buy = priceMatrix[i][0]
    best_buy_day = 0

    for j in range(1, totalDays):
        # Update best_buy using cooldown constraint
        if j - cooldownPeriod - 1 >= 0:
            candidate = priceMatrix[i][j] - dp[j - cooldownPeriod - 1]
        else:
            candidate = priceMatrix[i][j]

        if candidate < best_buy:
            best_buy = candidate
            best_buy_day = j

        # Compute profit if we sell today
        profit = priceMatrix[i][j] - best_buy

        # Option 1: skip today
        if dp[j - 1] >= dp[j]:
            dp[j] = dp[j - 1]
            txnTrack[j] = txnTrack[j - 1]
            # Option 2: sell today
            if profit > 0:
                if j - cooldownPeriod - 1 >= 0:
                    prevTxns = txnTrack[j - cooldownPeriod - 1]
                else:
                    prevTxns = []

                if j - cooldownPeriod - 1 >= 0:
                    newProfit = profit + dp[j - cooldownPeriod - 1]
                else:
                    newProfit = profit

                if newProfit > dp[j]:
                    dp[j] = newProfit
                    txnTrack[j] = prevTxns + [(i, best_buy_day, j)]

return txnTrack[totalDays - 1]
```